A.E.T.H.E.R — MANUAL STEPS

Everything you (Rayyan & Ashitha) must do by hand to take the repo from "cloned" to "fully working demo"

> **Read this once end-to-end before starting.** The repo already contains all the code, the Gazebo world + drone model, configs, launch files, the offline demo, and the docs. What you cannot do from Windows — install/run ROS2 + Gazebo + OpenVINS, train nothing (there is no training), record real trajectories, and (optionally) model a prettier drone in 3D — is listed here in order, with copy-paste commands and go/no-go gates.
>
> **Build window:** 8–11 June (the 12th is the 3-hour demo). **The #1 risk is Phase 1 (the environment).** Do it first.

✅ Verified platform status (tested on the dev machine, 10 Jun) — ALL THREE TIERS MEASURED

Verified working on Ubuntu 26.04 / ROS2 Lyrical (WSL2): - **T1 — live integrity demo:** `replay → integrity_monitor → degradation_manager → cockpit`. Camera-kill → `/nav/state` `NOMINAL→INERTIAL→NOMINAL`, `/nav/trust` `1.0→0.02`, bound blooms ~8× and recovers. **Measured (live ROS2): coverage 97.6 % emergent, ANEES 3.05, detection 1.18 s.** - **T3 — Gazebo 3D sim, physics-real flight:** the vendored **X3 multicopter** (4 rotor `MulticopterMotorModel`s + `MulticopterVelocityControl`) flies the full 200 m textured tunnel under **closed-loop guidance** (`flight_director`: feedforward + P feedback + yaw hold on simulator ground truth — the autopilot is not the system under test). x → 199.8 m, centered ±0.58 m; the IMU is **alive** (cruise accel σ 0.287 m/s² vs 0.002 on a kinematic rig). - **T2 — OpenVINS stereo-MSCKF on OUR sim** (via the `aether/openvins:jazzy` Docker container): golden bag (6,382 stereo pairs + 64 k IMU samples) → `ov_msckf` → **0.219 % terminal drift over 202.2 m** (DP7 gate < 1.5 % — beaten 6.8×), RMS ATE 0.202 m. The integrity layer also runs live against the real VIO output in the same container (`scripts/docker_run_integrity_chain.sh`).

Why Docker for OpenVINS: it does **not** build on 26.04/Lyrical yet (CMake 4 dropped the old `cmake_minimum_required`; Boost 1.90 made `system` header-only; `ament_target_dependencies` was removed). The pinned `ros:jazzy` image (`docker compose build openvins`, recorded SHA at `/OPENVINS_SHA`) makes T2 reproducible anywhere — including the GitHub Actions runner, which colcon-builds the whole workspace on `ros:jazzy` every push.

Demo recipes (the three rungs):

```
# rung 1 — live integrity beat (no Docker needed):
./scripts/run_replay_demo.sh           # + rviz2 -d src/aether_bringup/config/aether.rviz
./scripts/run_demo.sh kill && ./scripts/run_demo.sh restore
# rung 2 — the full chain on the golden bag (Docker):
docker compose run --rm chain          # drift report -> results/drift_report.txt
# rung 3 — record a fresh golden flight (native Gazebo), then re-run the chain:
bash scripts/wsl_record_bag.sh && docker compose run --rm chain
```

Phase 0 — What already works *today*, on any laptop (no Ubuntu)

You can validate the whole integrity algorithm right now, on Windows/Mac, before touching Ubuntu:

```
pip install numpy scipy matplotlib pytest
cd offline_demo
python verify_constants.py     # audits k_op=2.4477, k_ffd=6.7374, lambda_md=45.0
python run_demo.py             # regenerates docs/figures/*.png + prints metrics
pytest test_core.py -q         # proves the bound covers the truth 95%+
```

This is the **same math** the ROS2 `integrity_monitor` runs live. If these pass, the core is sound and the only thing left is wiring it to a real VIO on Ubuntu.

Phase 1 — Environment (Day 0, the critical path)

> You need **Ubuntu 22.04 LTS**. Native dual-boot is strongly preferred (best GPU + timing for Gazebo). A cloud GPU box (Lambda/Paperspace, Ubuntu 22.04 image) is the fallback. WSL2 only works for the *offline/integrity* parts (no live Gazebo rendering).

1.1 Install Ubuntu 22.04 (dual-boot)

1. Download Ubuntu 22.04.x Desktop ISO; flash to USB with **Rufus** (Windows) or **balenaEtcher**.
2. In Windows: Disk Management → shrink the C: volume to free **~80 GB**.
3. Boot the USB → *Install Ubuntu alongside Windows* → keep both bootloaders.
4. After first boot, verify the GPU: `bash sudo ubuntu-drivers autoinstall # NVIDIA: installs the proprietary driver, then reboot nvidia-smi # NVIDIA glxinfo | grep "OpenGL renderer" # AMD/Intel (sudo apt install mesa-utils)` *Budget ~1 hour.*

1.2 One-shot install of ROS2 + Gazebo + OpenVINS

The repo ships an installer. From the cloned repo root:

```
git clone https://github.com/TheClazer/A.E.T.H.E.R.git aether && cd aether
chmod +x scripts/*.sh
./scripts/setup_ubuntu.sh          # ROS2 Humble + Gazebo Harmonic + ros_gz + OpenVINS (Release)
```

Then add the two sources to your shell:

```
echo "source /opt/ros/humble/setup.bash"      >> ~/.bashrc
echo "source ~/ws_ov/install/setup.bash"        >> ~/.bashrc
source ~/.bashrc
```

Budget ~1.5–2 hours (downloads + the OpenVINS Release build).

1.3 (Optional, Day 3) PX4 SITL — for flying the drone in sim

The drone model carries sensors + ground truth; **PX4 provides the actuation** (or use the pure-gz fallback in 1.5).

```
git clone https://github.com/PX4/PX4-Autopilot.git --recursive
bash ./PX4-Autopilot/Tools/setup/ubuntu.sh    # reboot after
make px4_sitl gz_x500                          # smoke test: an x500 quad spawns in Gazebo
```

1.4 EuRoC dataset (the safety net + Day-1 validation)

Download 2–3 sequences from the EuRoC MAV dataset (ASL ETH): **V1_01_easy**, **MH_01_easy**, **MH_03_medium** (held-out test). Convert each to a ROS2 bag (the `kalibr` / `euroc2bag` tooling or rosbags). *Budget: download time, ~3–5 GB.*

1.5 Build A.E.T.H.E.R

```
cd ~/aether
colcon build --symlink-install --packages-select aether_msgs   # interfaces FIRST
colcon build --symlink-install
source install/setup.bash
```

> If `ros_gz_sim` / `ros_gz_bridge` aren't found, you're missing Harmonic bridge: `sudo apt install ros-humble-ros-gzharmonic`.
>
> **Pure-gz fallback (no PX4):** the world + model run under plain `gz sim sim/worlds/tunnel.sdf`; drive the drone with a velocity command or a scripted pose. The VIO + integrity layer are identical — only the actuation source changes.

1.6 ⛳ Day-0 go/no-go gate (must be green before Day 1)

#	Check	Pass	Fallback
1	`ros2 doctor` + `gz sim --version` (Sim 8.x)	both OK	re-add apt repos; cloud GPU box
2	OpenVINS runs on a EuRoC bag; `/ov_msckf/odomimu` publishes	yes	pin Ceres/OpenCV via apt; rebuild Release

#	Check	Pass	Fallback
3	`python eval/compute_drift.py gt.txt est.txt` on EuRoC	drift < 1.5%	check `--align_origin`, TUM timestamps
4	`colcon build` of A.E.T.H.E.R clean	yes	build `aether_msgs` first

Phase 2 — The floor (Days 1–2): all four DP7 deliverables

1. **OpenVINS on EuRoC** → record `/ov_msckf/odomimu`, convert, evaluate: `bash python eval/odom_to_tum.py <euroc_bag> /ov_msckf/odomimu est.txt python eval/odom_to_tum.py <euroc_bag> /aether/ground_truth gt.txt # or EuRoC GT python eval/compute_drift.py gt.txt est.txt`
2. **OpenVINS on the Gazebo tunnel** → in `src/aether_bringup/launch/vio.launch.py`, uncomment the `ov_msckf` node, then: `bash ./scripts/run_floor.sh` Fly a ~200 m pass; record `/ov_msckf/odomimu` and `/aether/ground_truth`.
3. **Performance report figures**: `bash python eval/make_report_figures.py gt.txt est.txt # measured trajectory python eval/compute_drift.py gt.txt est.txt # the headline numbers`
4. **Document the workspace** (deliverable #4 — already mostly done): per-package READMEs are the topic contracts; `rqt_graph` and `ros2 topic list -t` exports go into the report.

> **End of Day 2 = FLOOR LOCKED.** Tag it: `git tag floor-locked && git push origin floor-locked`.

Phase 3 — The live demo (Days 3–4): the breathing bound

1. Launch the full stack + RViz + HUD: `bash ros2 launch aether_bringup aether.launch.py use_sim:=true rviz2 -d src/aether_bringup/config/aether.rviz streamlit run src/aether_health_cockpit/aether_health_cockpit/streamlit_app.py`
2. Run the **kill-the-camera** beat (the gasp): `bash ./scripts/run_demo.sh # prints the beat sheet ./scripts/run_demo.sh kill # blank the stereo stream → trust RED < 0.5 s, bound blooms ./scripts/run_demo.sh restore # vision back → bound contracts, trust GREEN` Watch `/nav/trust`, `/nav/state`, `/nav/integrity_bound`, and the RViz ellipse chasing-and-containing the truth marker.
3. **Record the golden run** (rung-2 safety net): `ros2 bag record -a -o bags/aether_demo`. It replays identically with `use_sim:=false`.
4. **Backup video** (rung 3): screen-record the beat once it's clean. Rehearse to **5/5** clean runs.
5. **(Stretch, offline only)** the full per-feature RAIM-slope / observability-`D` math — compute on a dumped bag in a notebook, present as one backup slide. **Never** put the C++ fork on the live critical path.

> **End of Day 4:** `git tag demo-final && git push origin demo-final`.

Phase 4 — 3D design (OPTIONAL — the model already works without it)

You do NOT need to model anything in 3D to have a working demo. The drone in `sim/models/aether_drone/model.sdf` is built from primitives (boxes + cylinders) and already carries the stereo pair, the HG4930-class IMU, and the ground-truth publisher. The tunnel in `sim/worlds/tunnel.sdf` is parametric boxes. Both load and run as-is.

If you want a prettier drone or a richer tunnel for the demo video, here is the optional path:

4.1 Model a drone mesh in Blender (optional)

1. Install **Blender** (free). Model or download a quadrotor (keep it < ~50k tris).
2. Set the origin to the IMU location; **+X = forward, +Z = up** (ROS/Gazebo convention).
3. Export as **`.dae` (Collada)** or `.stl` into `sim/models/aether_drone/meshes/`.
4. In `model.sdf`, replace the `base_link` `<visual><geometry><box>...` with: `xml <visual name="v"> <geometry><mesh> <uri>model://aether_drone/meshes/drone.dae</uri><scale>1 1 1</scale></mesh></geometry> </visual>` **Keep the `<collision>` as a simple box** (a mesh collision is slow and unnecessary). Keep the `<sensor>` and `<plugin>` blocks unchanged.

4.2 A richer tunnel (optional)

- Add wall **textures** for more visual features (better tracking): in each wall `<material>`, point to a PBR material or a textured `<mesh>` instead of a flat box. High-frequency texture = more features = healthier nominal tracking.
- Or model a curved/branching tunnel in Blender, export `.dae`, and `<include>` it as a static model in `tunnel.sdf`.
- **Do not** over-detail — features and a clear 200 m path matter, polygon count does not.

4.3 Verify after any 3D change

```
gz sim sim/worlds/tunnel.sdf            # loads without errors?
ros2 topic echo /aether/ground_truth    # ground truth still streams?
```

Phase 5 — GitHub workflow

The repo is already on GitHub at `github.com/TheClazer/A.E.T.H.E.R` (public). To push your build progress:

```
git add -A && git commit -m "build: <what you did>"
git push origin main
git tag floor-locked && git push origin floor-locked   # at the milestones
```

CI (`.github/workflows/ci.yml`) lints and runs the offline integrity tests on every push — keep it green.

The checklist (print this)

- [ ] **Day 0** — Ubuntu 22.04; `setup_ubuntu.sh` ; EuRoC; `colcon build` clean. **Gate 1–4 green.**
- [ ] **Day 1–2** — OpenVINS on EuRoC + Gazebo; drift < 1.5%; report figures; workspace documented. **FLOOR LOCKED.**
- [ ] **Day 3–4** — integrity layer live; kill-camera demo; RViz + HUD; golden rosbag; backup video; 5/5 rehearsals.
- [ ] **(optional)** 3D drone/tunnel mesh in Blender.
- [ ] **12 Jun** — boot Ubuntu, `colcon build` , run one rung of the ladder, present.

> *"When the camera dies, our drift bound still covers the true position 95%+ of the time — and we know within half a second."*